



Ciência da Computação
Estrutura de Dados

A Linguagem C:
Características da Linguagem
Comando Básicos

Aldo Moura
aldo.moura@souunit.com.br

Roteiro



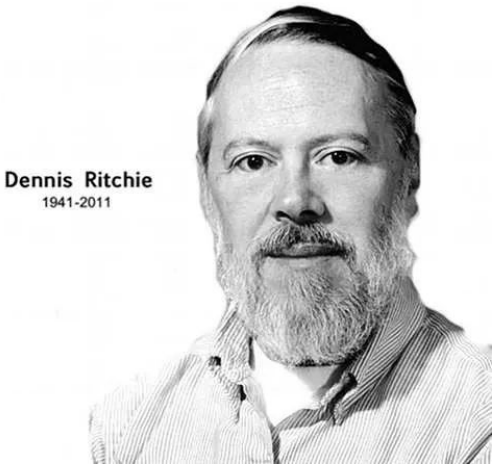
- Linguagem C
- Estrutura Básica
  - Editor de código / IDE
  - Função
  - Função main()
  - Comentário
- Saída de dados: prinf()
- Constantes e Variáveis
- Atribuição de dados
- Entrada de dados
- Conversão de tipo
- Visibilidade de variáveis

Linguagem C - Origem

Criada por Dennis Ritchie em 1972, originalmente escrito em Assembly.

Foi baseada na linguagem B (desenvolvida por Ken Thompson), que por sua vez era uma versão reduzida da BCPL.

É considerada a "linguagem mãe" de outras linguagens, como: Java, C++ e C#.



Linguagem C - Principais Características

- **De propósito geral**:
  Permite construir softwares para diversos aspectos, como: sistemas operacionais, drivers de dispositivos, desenvenvolvimento. web e mobile, ciência de dados, jogos, embarcados, automação e IoT.

- **Compilada**:
  O código fonte é traduzido para código de máquina, gerando um arquivo executável.

- **Imperativa**:
  Orienta o computador **como** realizar uma tarefa, através de uma sequência explicita de comandos.

- **Estruturada**:
  Utilização intenso de estrutura condicional (if e else) e de estrutura de repetição (for, while).

Linguagem C - Principais Características

- **Procedural**:
  Organiza o código blocos lógicos (funções).

- **Tipagem estática**:
  As variáveis precisam ter seus tipos declarados, pois são verificados em tempo de compilação.

- **Acesso de baixo nível a memória**:
  É capaz de manipular diretamente a memória do computador.

- **Case sensitive:**
  Distingue letras maiúsculas de minúsculas.
  Obs.: todas as palavras-chave são escritas em minúsculo.

Editor de Código / IDE

A edição de um código fonte em C pode ser realizada em qualquer editor de texto simples, por exemplo, o bloco de notas, requerendo que depois o código seja compilado.

Para facilitar as atividades de desenvolvimento de aplicação (edição do código, compilação, teste, depuração e execução) são utilizados os ambientes de desenvolvimento integrado (IDE - Integrated Development Environment

Exemplos de IDEs para a linguagem C:

DEV C++;

Falcon C++;

Estrutura de uma Função

```
<tipo_retorno> <nome_da_função> ([lista_parâmetros])
{
   // corpo da função
  [return <dado_retornado>]
}
```

Onde:
tipo de retorno: Tipo de dados que a função deve retornar ou void, se não tiver retorno.
nome_da_função: Nome atribuído pelo programador, que servirá de base para que a função seja chamada a execução.
lista parâmetros:

- Variáveis locais para representar cada um dos dados de entrada, necessários a execução da função.
- Se não houver parâmetros, se coloca apenas os parênteses.
- Havendo mais de um parâmetro eles devem ser separados por vírgula.
- Cada parâmetro deve definir um nome e seu respectivo tipo de dado

Função Obrigatória: main()

A função main é o ponto de entrada obrigatório, pois o sistema operacional chama esta função para iniciar a execução do programa.

Ela retorna um int. Por convenção é retornado 0 para indicar sucesso e valores diferentes de zero para indicar erros.

```c
#include <stdio.h>
int main()
{
    printf("Hello World");
    return 0;
}
```

O comando #include é uma diretiva que insere o conteúdo de um arquivo de cabeçalho (geralmente arquivos .h) no código-fonte antes da compilação. No exemplo acima, as funções pré-definidas da biblioteca stdio.h, como printf e scanf, poderão ser utilizadas.

Comentário de Linha

Comentários são as linhas do programa que deverão ser ignoradas pelo compilador. Isto é feito através da utilização da **//** no início do comentário.

Comentando Linhas

```
int main()
{
    //Esta linha será ignorada pelo compilador C
    printf("Olá mundo!");
    return 0;
}
```

Comentando
Parte de uma Linha

```
int main()
{
    printf("Olá mundo!");   //Assim também funciona
    return 0;
}
```

Comentário de Bloco

Comentário de blocos de textos é feito utilizando **/*** no início do bloco e ***/** no final do bloco.

```c
int main()
{
    /*
      Este bloco de linhas será ignorado
      pelo compilador C
    */
    printf("Olá mundo!");
    return 0;
}
```

Saída de Dados – função **printf**

Apresenta na tela, determinados dados (constantes, variáveis ou expressões) de acordo com um determinado formato. Para utilizar o printf, é necessário a inclusão da biblioteca <stdio.h>.

```
printf("saida_1[especificador_1] [saida_n] [especificador_n]"
       [, dado_1, ..., dado_n])
```

```
#include <stdio.h>

int main()
{
    printf("Hello World");
    return 0;
}
```

Saída de Dados – função **printf**

```
printf("saida_1[especificador_1] [saida_n] [especificador_n]"
       [, dado_1, ..., dado_n])
```

```
printf("Valor");  // Mostra o texto Valor
printf("2 + 6");  // Mostra o texto 2 + 6
```

Saída de Dados – função **printf**

```
printf("saida_1[especificador_1] [saida_n] [especificador_n]"
       [, dado_1, ..., dado_n])
```

Onde: especificador: especifica o formato do dado que será substituído no texto
%c: char / %d: int / %f: float ou double
dado: dado que comporá o texto a ser exibido

```
printf("Valor: %d", 500);          // Mostra o texto Valor: 500
printf("A soma é: %d", 2 + 6);     // Mostra o texto A soma Ú: 8
```

O problema da acentuação pode ser resolvido utilizando a biblioteca <locale.h> e a função setlocale.

```
#include <stdio.h>
#include <locale.h>
int main()
{
    setlocale(LC_ALL,"portuguese");
    printf("A soma é: %d", 2 + 6);
    return 0;
}
```

Saída de Dados – função **printf**

```
printf("saida_1[especificador_1] [saida_n] [especificador_n]"
       [, dado_1, ..., dado_n])
```

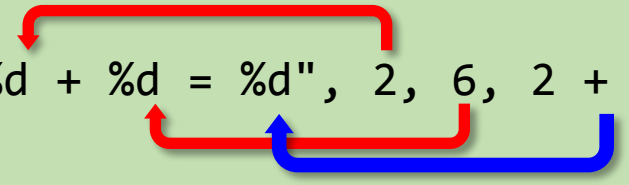
```
printf("Valor: %d reais", 500);         // Mostra o texto Valor: 500 reais
printf("Juros: %f porcento", 2.3);      // Mostra o texto Juros: 2,300000 porcento
printf("Juros: %.1f porcento", 2.3);    // Mostra o texto Juros: 2,3 porcento
```

função **printf - Especificadores**

%d	Número inteiro (int). Também pode ser usado %i como equivalente a %d.
%u	Número decimal natural (unsigned int), ou seja, sem sinal.
%o	Número inteiro representado na base octal. Exemplo: 41367 (corresponde ao decimal 17143).
%x	Número inteiro representado na base hexadecimal. Exemplo: 42f7 (corresponde ao decimal 17143). Se usarmos %X, as letras serão maiúsculas: 42F7.
%X	Hexadecimal com letras maiúsculas
%f	Número decimal de ponto flutuante. No caso da função printf, devido às conversões implícitas da linguagem C, serve tanto para **float** como para **double**. No caso da função scanf, %f serve para **float** e %lf serve para **double**.
%e	Número em notação científica, por exemplo 5.97e-12. Podemos usar %E para exibir o E maiúsculo (5.97E-12).
%E	Número em notação científica com o "e"maiúsculo
%g	Escolhe automaticamente o mais apropriado entre %f e %e. Novamente, podemos usar %G para escolher entre %f e %E.
%p	Ponteiro: exibe o endereço de memória do ponteiro em notação hexadecimal.
%c	Caractere: imprime o caractere que tem o código ASCII correspondente ao valor dado.
%s	Sequência de caracteres (string, em inglês).
%%	Imprime um %

Saída de Dados – função **printf**

```
printf("saida_1[especificador_1] [saida_n] [especificador_n]"
       [, dado_1, ..., dado_n])
```



```
printf("A soma de %d + %d = %d", 2, 6, 2 + 6);    // Mostra o texto A soma de 2 + 6 = 8


printf("Produto: %s - Valor: R$ %.2f", "Camisa", 189.99);
                                        // Mostra Produto: Camisa - Valor: R$ 189,99
printf("Juros: %.1f %%", 2.3);          // Mostra o texto Juros: 2,3 %
```

Saída de Dados – Caracteres de Escape

São caracteres com a propriedade de executar determinadas operações

\n	Quebra de linha (*line feed* ou LF)
\t	Tabulação horizontal
\b	Retrocede o cursor em um caractere (backspace)
\r	Retorno de carro (*carriage return* ou CR): volta o cursor para o começo da linha sem mudar de linha
\a	Emite um sinal sonoro
\f	Alimentação de formulário (*form feed* ou FF)
\v	Tabulação vertical (em impressoras)
\"	Aspas
\'	Apóstrofos
****	Barra invertida
\0	Caractere nulo (caractere de valor zero, usado como terminador de strings)
\N	O caractere cuja representação octal é N (dígitos de 0 a 7)
\xN	O caractere cuja representação hexadecimal é N (dígitos de 0 a 9 e de A a F)

Saída de Dados – função **printf**

```c
#include <stdio.h>

int main()
{
    printf("\aLinguagem \'C\'\n");
    printf("Principal\\Essencial linguagem de programação.\tOK");
    return 0;
}
```

```
Linguagem 'C'
Principal\Essencial Linguagem de programação.   OK
```

Saída de Dados – função **printf**

```
#include <stdio.h>
#include <locale.h>
#include <stdlib.h>

int main()
{
    setlocale(LC_ALL, "portuguese");
    printf("Análise e Desenvolvimento de Sistemas\n\n");
    printf("Tecle algo para prosseguir\n");
    getchar();
    printf("Estrutura de dados\n");
    system("pause"); // Requer o uso da biblioteca stdlib.h
    printf("Linguagem C\n");
    return 0;
}
```

Principais Tipos Básicos de Dados

Inteiro:

- int: 2 bytes (-32768 até +32767) ou 4 bytes (-2147483648 até 2147483647).

Ponto flutuante:

- float: 4 bytes
- double: 8 bytes

Caractere:

- char: armazena um caractere alfabético

Booleano:

- boolean: (1 bit) 0 representa false; valor diferente de 1 representa true

Declaração de Constante

```
const <tipo_dado> <ID_CONSTANTE> = <literal>
```

```c
#include <stdio.h>
#include <locale.h>

int main()
{
    setlocale(LC_ALL, "portuguese");
    const float PI = 3.14159265;
    float raio = 5.3;
    float area_circulo = PI * raio * raio;
    float area_esfera = 4 * PI * raio * raio;
    float volume_esfera = 4/3 * PI * raio * raio * raio;
    return 0;
}
```



Boa prática: o nome identificador da constante em letras maiúsculas

Declaração de Constante

```
#define <ID_CONSTANTE> <literal>
```

```
#include <stdio.h>
#include <locale.h>

#define PI 3.14159265

int main()
{
    setlocale(LC_ALL, "portuguese");
    float raio = 5.3;
    float area_circulo = PI * raio * raio;
    float area_esfera = 4 * PI * raio * raio;
    float volume_esfera = 4/3 * PI * raio * raio * raio;
   return 0;
}
```

Variável

```
tipo_dado <id_variável_1> [, <id_variavel_n>]
```

```
int numero;
float salarioBruto;
double indiceInflacaoMensal;
char tipoCurso;
int numero1, numero2
float salarioBruto, salarioLiquido;
```



A declaração pode ocorrer em qualquer parte do programa.
Boa prática: Utilizar a notação lowerCamelCase para nomear identificadores de variáveis.
Uma variável armazena apenas um valor por vez.

Atribuição de dados a uma Variável

1. Durante a declaração da variável

```
tipo_dado <id_variável_1> [ = <dado>]
```



Exemplos:

```
int idade = 18;
float nota1 = 7.8, nota2 = 9.5;
float media = (nota1 + nota2) / 2;
char letra = 'A';
```

FICA A DICA

O conteúdo atribuído a variável pode ser literal, uma constante, uma variável ou uma expressão.

Atribuição de dados a uma Variável

2. Posterior a declaração da variável

```
<nome_variavel> = <conteúdo>
```

Exemplos:

```
int idade;
float nota1, nota2, media;
char letra;

idade = 18;
nota1 = 7.8, nota2 = 9.5;
media = (nota1 + nota) / 2;
letra = 'A';
```

Atribuição de dados a uma Variável

3. Atribuição por entrada via teclado. Função **scanf**

```
scanf("<especificador_1> [especificador_n]", <&var_1> [<&var_2>... <&var_2>])
```

Onde:

especificador: especifica o tipo do dado que será atribuído

%c: char / %d: int / %f: float ou double

var = variável que armazenará o dado informado.

```c
#include <stdio.h>

int main(void) {
  int num1, num2, soma;

  scanf("%d", &num1);
  scanf("%d", &num2);

  soma = num1 + num2;
  printf("Soma: %d", soma);
  return 0;
}
```

O valor digitado será atribuído a um endereço de memória. O símbolo & captura o endereço de memória da variável referenciada.

Tipo de Dados String

É uma sequência de caracteres (tipo char), geralmente utilizada para representar palavras, frases ou textos de um programa. Na linguagem C, as strings são tratadas como um conjunto de caracteres.

Ex:

A string "BRASIL" é, na verdade, ["B","R","A","S","I","L"]

Entrada de Dados do Tipo String - **scanf**

```c
#include <stdio.h>

int main()
{
    char nome[20];
    printf("Qual o seu nome? ");
    scanf("%s", nome);
    printf("\nMuito prazer, %s!", nome);
    return 0;
}
```

Obs.:

Não utilizar o operador & antes da variável na função scanf.

Entrada de Dados do Tipo String - **scanf**

```
scanf("< %tamanho><[^\n]>", <&var>)
```

```c
#include <stdio.h>

int main()
{
    char nome[20];
    printf("Qual o seu nome? ");
    scanf(" %20[^\n]", &nome);
    printf("Muito prazer, %s!\n", nome);
    return 0;
}
```

Obs.: Antes do especificador do tamanho deve ter um espaço.

Entrada de Dados do Tipo String - **fgets**

```
fgets("<variável>, <tamanho>, <entrada>)
```

Onde: Variável – Espaço de memória onde será armazenada a string;

Tamanho – Quantidade máxima de caracteres na string;

Entrada – Objeto que receberá a entrada de dados.

```c
#include <stdio.h>

int main()
{
    char nome[12];
    printf("Qual o seu nome? ");
    fgets(nome, 12, stdin);
    printf("Muito prazer, %s!\n", nome);
    return 0;
}
```

Obs. O objeto de entrada stdin, indica que o dado será recebido do dispositivo padrão de entrada.